SIMATSENGINEERING

Department of Information Technology

DSA01–Object-OrientedProgrammingwithC++

ASSIGNMENT3–CO3

CONSTRUCTORS, DESTRUCTORS AND OPERATOR OVERLOADING IN C++

CO3:Applyobject-orientedprogrammingprinciplesandC++constructssuchasdatatypes,operators and control structures to design and write structured OOP-based programs.

Student Name	D GOVARDHAN RAJU	**Register Number**	192525125
Department / Section	AI&DS	**Academic Year / Semester**	2025
FacultyName	**AMIRTHA S**	**Natureof Assignment**	Individual
Duration	1Week	**TotalMarks**	50
Date of Submission	06-11-2026	**Date of Evaluation**	
GitHub Repository	192525125-DSA0114	**Compiler**	g++13.3.0,-std=c++17-Wall-Wextra

MARK SHEET(Rubric-basedEvaluation)

Task	TaskTitle	Max	Marks Awarded	Remarks
1	Object-Oriented Analysis	4		
2	Tokens, Keywords, Identifiers and Constants	3		
3	DataTypesandCompatibility	4		
4	ImplicitConversions	4		
5	OperatorsandPrecedence	4		
6	ControlStructures	4		
7	ProceduralvsObject-Oriented	4		
8	InputandOutput	4		
9	ImplementationandTesting	4		
10	ExpectationvsExecution	4		
11	FinalRecommendation	4		
12A	Re-evaluate:Impact Analysis	2		
12B	Re-evaluate: Evaluation of Original Design	2		
12C	Re-evaluate: Adaptation or Redesign	2		
12D	Re-evaluate: Evidence-Based Defence	2		
	TOTAL	50		Percentage:

OVERALLREMARKS

D GOVARDHAN RAJU		
Student Signature	**Faculty Signature**	**Program Director**

OBJECT-ORIENTEDPROGRAMMINGWITHC++

ASSIGNMENT 3 – CO3

CO3: Apply object-oriented programming principles and C++ constructs such as data types, operators and control structures to design and write structured OOP-based programs – the object-oriented paradigm and itsbasic principles, tokens,keywords, identifiers, constants, data types,type compatibility,variables,operators,implicitconversions,operatorprecedenceandcontrolstructures.

NatureofAssignment: Individual
Duration:1Week
TotalMarks:50
Submission Date: _____
Bloom'sLevelsAssessed:K5–Evaluate(everytaskinthisassignmentisatK5level)

AssignmentTitle: CONSTRUCTORS, DESTRUCTORS AND OPERATOR OVERLOADING IN C++ (Total Marks - 50)

MandatoryExecution and GitHubSubmission

- Everystudent must individuallyexecuteeverytaskinthisassignment,includingall"Complete the Code" programs.
- Create a GitHub repositorynamed DSA01-<RegisterNumber>-Assignment1 and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keeponefolder foreachtask(Task01,Task02…),containingtheC++sourcefiles,input files and output screenshots.
- IncludeaREADME.mdwithyourname,registernumber,compilerandversion,theexact g++ commandusedtocompile, thestepstoruneachprogram, andashort noteoftheresultofeach task.
- Commit your work progressively – at least one commit for each task. A single upload on the last day will not be accepted.
- SubmittheGitHublinkbelowandonthecover pageofyour report.

GitHub Repository Link: _____

Howto AnswertheAdditionalQuestions

Complete the Code: Copythe given code into your repository, replace everyTODO with working codewithoutchangingtherestoftheprogram,executeitandcompareyouroutputwiththeexpected output.

Predict the Output: Writethe exact output ofthe givenprogrambefore running it. Thenrun it and explaineverydifferencebetweenyourpredictionandtheactualoutput,usingtheC++rulesinvolved. Predictions written after running the program will not be credited.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Scenario—NammaMetroSmart-CardFare System

A city metro rail divides its network into three zones – Inner, Middle and Outer. The fare is Rs 10, Rs20 or Rs30 according tothe farthest zonetravelled, witha20% surchargeduring peak hours(8–10 AM and 5–8 PM). Passengers use a stored-value smart card, and a journey is allowed only if at least Rs 50 remains on the card after the fare is deducted. Students and senior citizens receive a concession.

The first version ofthe gatesoftware was written in a hurry. Some fares are off bya paisa, the peak checkaccepts10AMaspeakand8PMasoff-peakinconsistently,anegativebalanceonceappeared on a card, and the fare values are typed in several places.

Thenumberofjourneysprocessed maygrow:1station(20,000journeys aday)→1 line(500,000a day) → whole network (3,000,000 a day).

You are required to analyse the problem, identify its objects, choose suitable C++ data types, constants,operatorsandcontrolstructures,implementthefarelogicandvalidateit,sothateveryfare and balance is exact.

Design Challenge:

The metro is adding a fourth zone, a weekend discount and monthly passes. Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

INTRODUCTION

Constructors, destructors and operator overloading are important concepts in C++ object-oriented programming. Constructors are special member functions used to initialise objects when they are created. Destructors automatically release resources when objects are destroyed. Operator overloading allows programmers to define how operators such as +, -, ==, ++ and << work with user-defined classes.

These concepts help develop reliable, reusable and efficient programs. They are particularly useful when a class manages dynamically allocated memory or represents real-world entities such as money, time, text, complex numbers and booking records.

OBJECTIVES

1. To understand different types of constructors in C++.
2. To implement default, parameterised, overloaded and copy constructors.
3. To understand shallow copying and deep copying.
4. To implement destructors for automatic memory management.
5. To overload arithmetic, comparison and stream operators.
6. To understand prefix and postfix operator overloading.
7. To analyse alternative designs using time and space complexity.

Q1 – CONSTRUCTORS AND CONSTRUCTOR OVERLOADING

1.1 Problem Description

A library maintains book records in its computer system. A book record may be created with complete details, with only a title when a book is donated, or as an empty record that will be updated later.

The program must use different constructors to initialise book objects. It must also prevent invalid values, such as a negative number of copies.

1.2 Concepts Used

- Default constructor
- Parameterised constructor
- Constructor overloading
- Member initializer list
- Constructor delegation
- Explicit constructor

- Data validation

1.3 Algorithm

1. Start the program.
2. Create a Book class.
3. Declare private data members for title, author and number of copies.
4. Define a default constructor to initialise an empty book.
5. Define a parameterised constructor to initialise all details.
6. Define an overloaded constructor for donated books.
7. Use constructor delegation to avoid repeated initialisation.
8. Validate the number of copies.
9. Create objects using different constructors.
10. Display the details of each book.
11. Stop the program.

1.4 C++ Program

```
#include <iostream>
#include <string>
using namespace std;

class Book {
private:
    string title;
    string author;
    int copies;

public:
    Book() : Book("Unknown", "Unknown", 0) {}

    Book(string t) : Book(t, "Unknown", 1) {}

    Book(string t, string a, int c)
        : title(t), author(a), copies(c >= 0 ? c : 0) {}

    void display() const {
        cout << "Title: " << title << endl;
        cout << "Author: " << author << endl;
        cout << "Copies: " << copies << endl;
        cout << "------------------" << endl;
    }
};

int main() {
    Book b1;
    Book b2("C++ Programming");
    Book b3("Data Structures", "Mark Allen", 5);

    b1.display();
    b2.display();
    b3.display();

    return 0;
}
```

1.5 Expected Output

```
Title: Unknown
Author: Unknown
Copies: 0
------------------
Title: C++ Programming
Author: Unknown
Copies: 1
------------------
```

```
Title: Data Structures
Author: Mark Allen
Copies: 5
------------------
```

1.6 Explanation

The default constructor creates an empty book record. The single-argument constructor creates a donated book with a title and one copy. The parameterised constructor accepts the title, author and number of copies.

Constructor delegation is used to call another constructor of the same class. This reduces code duplication and makes initialisation consistent.

A member initializer list initialises data members directly before the constructor body executes. It is generally preferred over assigning values inside the constructor body.

Members are initialised in the order in which they are declared in the class, not in the order in which they appear in the initializer list.

1.7 When Is a Default Constructor Provided?

The compiler provides a default constructor when no constructor has been declared by the programmer. If a parameterised constructor is declared, the compiler does not automatically provide a default constructor.

For example:

```
class Sample {
public:
    Sample(int x) {}
};
```

In this example, `Sample obj;` produces a compilation error because no default constructor is available.

1.8 Use of Explicit Constructor

The `explicit` keyword prevents unwanted implicit conversion.

```
class Book {
public:
    explicit Book(string title) {}
};
```

Without `explicit`, the following conversion may be allowed:

```
Book b = string("C++");
```

With `explicit`, this implicit conversion is not allowed. The programmer must write:

```
Book b(string("C++"));
```

1.9 Alternative Concept – Default Arguments

Instead of multiple overloaded constructors, a single constructor can use default arguments.

```
#include <iostream>
#include <string>
```

```
using namespace std;

class Book {
    string title, author;
    int copies;

public:
    Book(string t = "Unknown",
         string a = "Unknown",
         int c = 0)
        : title(t), author(a), copies(c >= 0 ? c : 0) {}

    void display() const {
        cout << title << " | " << author
             << " | " << copies << endl;
    }
};

int main() {
    Book b1;
    Book b2("C++ Programming");
    Book b3("Data Structures", "Mark Allen", 5);

    b1.display();
    b2.display();
    b3.display();

    return 0;
}
```

1.10 Complexity Analysis

Operation	Time Complexity	Space Complexity
Object creation	O(1)	O(1)
Member initialisation	O(1)	O(1)
Display details	O(1)	O(1)

Both overloaded constructors and a constructor with default arguments have O(1) object creation complexity for this class. Constructor overloading makes the different creation situations clearer, while default arguments reduce the number of constructor definitions.

1.11 Conclusion

Constructor overloading allows objects to be created in different ways. Member initializer lists provide direct and efficient initialisation. Constructor delegation avoids repeated code, and validation prevents invalid data from entering the object.

Q2 – COPY CONSTRUCTORS: SHALLOW AND DEEP COPIES

2.1 Problem Description

A hospital maintains prescription records containing dynamically allocated arrays of medicine names. When a prescription is copied, the original and copied objects should have independent medicine records.

A shallow copy copies only the pointer address, causing both objects to access the same memory. This may result in unexpected changes and double deletion. Deep copying solves this problem by allocating separate memory for each object.

2.2 Concepts Used

- Copy constructor
- Copy assignment operator
- Shallow copy
- Deep copy
- Destructor
- Dynamic memory allocation
- Rule of Three
- Self-assignment

2.3 Algorithm

1. Start the program.
2. Create a Prescription class.
3. Declare an integer to store the number of medicines.
4. Allocate a dynamic array of medicine names.
5. Implement a copy constructor.
6. Allocate new memory during copying.
7. Copy every medicine into the new array.
8. Implement the copy assignment operator.
9. Check for self-assignment.
10. Release the old memory before replacing it.
11. Implement the destructor to release allocated memory.
12. Create original and copied prescription objects.
13. Modify one copy and verify that other objects remain unchanged.
14. Stop the program.

2.4 C++ Program – Deep Copy

```
#include <iostream>
#include <string>
using namespace std;

class Prescription {
    int n;
    string* medicines;

public:
    explicit Prescription(int count)
        : n(count), medicines(new string[count]) {}

    // Copy constructor
    Prescription(const Prescription& other)
        : n(other.n), medicines(new string[other.n]) {

        for (int i = 0; i < n; i++) {
            medicines[i] = other.medicines[i];
```

```
        }
    }

    // Copy assignment operator
    Prescription& operator=(const Prescription& other) {
        if (this != &other) {
            string* temp = new string[other.n];

            for (int i = 0; i < other.n; i++) {
                temp[i] = other.medicines[i];
            }

            delete[] medicines;

            medicines = temp;
            n = other.n;
        }

        return *this;
    }

    // Destructor
    ~Prescription() {
        delete[] medicines;
    }

    void set(int i, const string& m) {
        medicines[i] = m;
    }

    string get(int i) const {
        return medicines[i];
    }
};

int main() {
    Prescription p(2);

    p.set(0, "Paracetamol");
    p.set(1, "ORS");

    Prescription q = p;

    q.set(1, "Cetirizine");

    Prescription r(1);
    r = p;

    p.set(0, "Ibuprofen");

    cout << p.get(0) << " " << p.get(1)
         << " | " << q.get(0) << " " << q.get(1)
         << " | " << r.get(0) << " " << r.get(1)
         << endl;

    return 0;
}
```

2.5 Expected Output

```
Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS
```

2.6 Explanation

The original prescription contains Paracetamol and ORS. The copy constructor creates a new prescription object with a separate medicine array.

When the second medicine in object `q` is changed to Cetirizine, the original object remains unchanged. Similarly, changing the first medicine in `p` does not affect `r`.

The copy assignment operator first checks whether the object is being assigned to itself. It then creates a temporary array, copies the medicines and releases the old array. This protects the existing object if allocation fails before the replacement is completed.

The destructor releases dynamically allocated memory using `delete[]`.

2.7 Shallow Copy vs Deep Copy

Shallow Copy	Deep Copy
Copies the pointer address	Copies the actual data
Both objects share memory	Each object has separate memory
Modification affects shared data	Modification remains independent
Can cause double deletion	Avoids shared ownership problems
Not suitable for owning raw pointers	Suitable when each object owns its memory

2.8 Rule of Three

The Rule of Three states that if a class requires a custom destructor, copy constructor or copy assignment operator, it usually needs all three.

1. Destructor – releases dynamically allocated memory.
2. Copy constructor – creates an independent copy.
3. Copy assignment operator – safely assigns one existing object to another.

These functions work together to manage resources correctly.

2.9 Three Situations Where Copy Constructor Is Called

1. When an object is initialised using another object.
2. When an object is passed to a function by value.
3. When an object is returned by value in situations where copy construction is performed.

Modern C++ compilers may apply copy elision, so a copy is not guaranteed in every return-by-value situation.

2.10 Alternative Concept – Using Vector

The medicine names can be stored using `std::vector<string>`. A vector automatically manages memory and supports independent copying.

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;

class Prescription {
    vector<string> medicines;

public:
    Prescription(int n) : medicines(n) {}

    void set(int i, string m) {
        medicines.at(i) = m;
    }

    string get(int i) const {
        return medicines.at(i);
```

```
    }
};

int main() {
    Prescription p(2);

    p.set(0, "Paracetamol");
    p.set(1, "ORS");

    Prescription q = p;
    q.set(1, "Cetirizine");

    cout << p.get(1) << endl;
    cout << q.get(1) << endl;

    return 0;
}
```

2.11 Complexity Analysis

Let n represent the number of medicines.

Operation	Time Complexity	Auxiliary Space
Shallow copy	O(1)	O(1)
Deep copy	O(n)	O(n)
Copy assignment	O(n)	O(n) temporary
Destructor	O(n) overall element destruction	O(1) additional

The vector implementation also performs an O(n) copy and uses O(n) storage for the copied medicines. Its advantage is automatic memory management.

2.12 Conclusion

Deep copying is necessary when a class owns dynamically allocated memory. It prevents two objects from sharing the same memory unintentionally. The Rule of Three provides a safe approach for resource management. Using `std::vector` is often simpler because it follows the Rule of Zero and automatically manages memory.

Q3 – DYNAMIC CONSTRUCTORS AND DESTRUCTORS

3.1 Problem Description

An auditorium requires a seat map for each event. Different events may have different numbers of rows and seats per row. The program must dynamically allocate the seat map, allow booking and cancellation, and release all memory when the event object is destroyed.

3.2 Concepts Used

- Dynamic constructor
- Dynamic memory allocation
- Two-dimensional array
- Destructor
- `new` and `delete`
- Stack and heap objects
- RAII
- Memory management

3.3 Algorithm

1. Start the program.
2. Create an Event class.
3. Store the number of rows and seats per row.
4. Dynamically allocate an array of row pointers.
5. Allocate a seat array for every row.
6. Initialise all seats as available.
7. Implement a booking function.
8. Implement a cancellation function.
9. Display the seat map.
10. Implement a destructor to release every row and the row-pointer array.
11. Create stack and heap event objects.
12. Destroy the objects and observe destructor execution.
13. Stop the program.

3.4 C++ Program

```
#include <iostream>
using namespace std;

class Event {
private:
    int rows, seats;
    char** seatMap;
    int booked;

public:
    Event(int r, int s)
        : rows(r), seats(s), seatMap(nullptr), booked(0) {

        if (rows > 0 && seats > 0) {
            seatMap = new char*[rows];

            for (int i = 0; i < rows; i++) {
                seatMap[i] = new char[seats];

                for (int j = 0; j < seats; j++) {
                    seatMap[i][j] = 'O';
                }
            }
        }
```

```
        cout << "Event created" << endl;
    }

    void book(int r, int s) {
        if (r < 0 || r >= rows || s < 0 || s >= seats) {
            cout << "Invalid seat" << endl;
            return;
        }

        if (seatMap[r][s] == 'O') {
            seatMap[r][s] = 'X';
            booked++;
            cout << "Seat booked" << endl;
        } else {
            cout << "Seat already booked" << endl;
        }
    }

    void cancel(int r, int s) {
        if (r < 0 || r >= rows || s < 0 || s >= seats) {
            cout << "Invalid seat" << endl;
            return;
        }

        if (seatMap[r][s] == 'X') {
            seatMap[r][s] = 'O';
            booked--;
            cout << "Booking cancelled" << endl;
        } else {
            cout << "Seat is not booked" << endl;
        }
    }

    void display() const {
        cout << "Seat Map:" << endl;

        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < seats; j++) {
                cout << seatMap[i][j] << " ";
            }
            cout << endl;
        }
    }

    ~Event() {
        if (seatMap != nullptr) {
            for (int i = 0; i < rows; i++) {
                delete[] seatMap[i];
            }

            delete[] seatMap;
        }

        cout << "Event closed. Booked seats: "
             << booked << endl;
    }
};

int main() {
    Event e1(2, 3);

    e1.book(0, 1);
    e1.book(1, 2);
    e1.display();

    Event* e2 = new Event(3, 2);
```

```
    e2->book(0, 0);
    e2->display();

    delete e2;

    {
        Event e3(1, 2);
        e3.book(0, 0);
    }

    return 0;
}
```

3.5 Expected Output

```
Event created
Seat booked
Seat booked
Seat Map:
O X O
O O X
Event created
Seat booked
Seat Map:
X O
O O
O O
Event closed. Booked seats: 1
Event created
Seat booked
Event closed. Booked seats: 1
Event closed. Booked seats: 2
```

3.6 Explanation

The constructor dynamically creates a two-dimensional seat map. The first allocation creates an array of row pointers, and the second allocation creates the seat array for each row.

The character `O` represents an available seat, while `X` represents a booked seat.

The booking function changes an available seat to booked. The cancellation function changes a booked seat back to available.

The destructor releases every row using `delete[]` and then releases the array of row pointers.

3.7 Constructor and Destructor Execution Order

- Local objects are destroyed automatically when their scope ends.
- Objects created inside a block are destroyed when the block ends.
- Objects created using `new` remain allocated until `delete` is used.
- Destructors for local objects execute in the reverse order of their construction.

3.8 What Happens if Delete Is Forgotten?

If dynamically allocated memory is not released, a memory leak occurs. The program continues to hold memory that is no longer needed.

Using `delete` instead of `delete[]` for an array allocated with `new[]` causes undefined behaviour. The correct form must be used.

3.9 Why Cannot a Destructor Be Overloaded?

A destructor has a fixed name, `~ClassName`, takes no parameters and has no return type. Therefore, a class can have only one destructor.

3.10 Alternative Concept – Single Contiguous Array

Instead of allocating each row separately, the complete seat map can be stored in one continuous block of memory.

The index for a seat can be calculated using:

```
index = row * seatsPerRow + seat
```

3.11 Complexity Analysis

Let r represent the number of rows and s represent seats per row.

Operation	Row-wise allocation	Single-block allocation
Allocation time	O(rs)	O(rs)
Seat access	O(1)	O(1)
Booking	O(1)	O(1)
Space	O(rs + r)	O(rs)
Number of allocations	r + 1	1

The single-block method uses fewer allocations and avoids storing row pointers. Both approaches provide constant-time seat access.

3.12 Conclusion

Dynamic constructors allow objects to allocate memory according to their requirements. Destructors automatically release resources when objects are destroyed. The RAII principle connects resource management to object lifetime and helps prevent memory leaks.

Q4 – OVERLOADING ARITHMETIC AND STREAM OPERATORS

4.1 Problem Description

A billing counter stores money in rupees and paise. The program must perform addition, subtraction and comparison operations using overloaded operators. It must also support input and output through stream operators.

4.2 Concepts Used

- Operator overloading
- Arithmetic operators
- Comparison operators
- Stream insertion operator
- Stream extraction operator
- Friend function
- Member function

4.3 Algorithm

1. Start the program.
2. Create a Money class.
3. Store the amount internally as paise.
4. Define a constructor to initialise rupees and paise.
5. Overload the addition operator.
6. Overload the subtraction operator.
7. Overload the equality and less-than operators.
8. Overload the compound addition operator.
9. Overload the stream insertion operator.
10. Display the calculated bill.
11. Stop the program.

4.4 C++ Program

```
#include <iostream>
#include <iomanip>
using namespace std;

class Money {
    long long paise;

public:
    Money(long long rupees = 0, int p = 0)
        : paise(rupees * 100 + p) {}

    Money operator+(const Money& other) const {
        Money result;
        result.paise = paise + other.paise;
        return result;
    }

    Money operator-(const Money& other) const {
        Money result;
        result.paise = paise - other.paise;
        return result;
    }

    Money& operator+=(const Money& other) {
        paise += other.paise;
        return *this;
    }
```

```
    bool operator==(const Money& other) const {
        return paise == other.paise;
    }

    bool operator<(const Money& other) const {
        return paise < other.paise;
    }

    friend ostream& operator<<(ostream& out,
                               const Money& m) {
        long long amount = m.paise;
        long long rupees = amount / 100;
        long long cents = amount % 100;

        if (cents < 0) {
            cents = -cents;
        }

        out << "Rs " << rupees << "."
            << setw(2) << setfill('0') << cents;

        return out;
    }

    friend istream& operator>>(istream& in, Money& m) {
        long long r;
        int p;

        in >> r >> p;

        if (in) {
            m.paise = r * 100 + p;
        }

        return in;
    }
};

int main() {
    Money a(120, 75);
    Money b(79, 25);

    Money c = a + b;

    cout << c << endl;
    cout << boolalpha << (c == Money(200)) << endl;

    Money d(0, 75);
    Money e(0, 50);

    cout << d + e << endl;

    return 0;
}
```

4.5 Expected Output

```
Rs 200.00
true
Rs 1.25
```

4.6 Explanation

The Money class stores the complete amount in paise. This avoids many floating-point precision problems associated with storing currency using `double`.

The + operator creates a new Money object containing the sum. The - operator creates a new object containing the difference. The += operator modifies the existing object and returns a reference to `*this`.

The equality operator compares the internal paise values. The stream insertion operator displays the amount in rupees and paise.

The << operator is implemented as a non-member function because the left operand is the output stream object, not a Money object.

4.7 How Is a + b Translated?

When the compiler sees:

```
Money c = a + b;
```

It treats the operation like:

```
Money c = a.operator+(b);
```

This is the member-function form of operator overloading.

4.8 Why Does + Return a New Object?

The + operator represents the creation of a new sum without modifying the original operands.

The += operator modifies the left-hand object. Therefore, it returns a reference to the current object to support chained assignments.

4.9 Alternative Concept – Named Functions

Instead of overloaded operators, named functions can be used.

```
#include <iostream>
using namespace std;

class Money {
    long long paise;

public:
    Money(long long r = 0, int p = 0)
        : paise(r * 100 + p) {}

    long long getPaise() const {
        return paise;
    }
};

Money add(const Money& a, const Money& b) {
    return Money(0, static_cast<int>(
        a.getPaise() + b.getPaise()));
}

bool equals(const Money& a, const Money& b) {
    return a.getPaise() == b.getPaise();
}

int main() {
    Money a(120, 75);
```

```
    Money b(79, 25);

    Money c = add(a, b);

    cout << equals(c, Money(200)) << endl;

    return 0;
}
```

4.10 Complexity Analysis

Operation	Time Complexity	Space Complexity
Addition	O(1)	O(1)
Subtraction	O(1)	O(1)
Comparison	O(1)	O(1)
Stream output	O(1)	O(1)

Operator overloading and named functions have the same Big-O complexity for these operations. Operator overloading improves readability by allowing natural mathematical expressions.

4.11 Engineering Decision

Multiplication of a Money object by a numeric quantity can be meaningful, such as calculating the cost of multiple items. Multiplication of one Money object by another Money object is generally not meaningful for ordinary billing calculations.

4.12 Conclusion

Operator overloading makes user-defined objects easier to use with familiar mathematical expressions. Storing currency as integer paise improves accuracy, and stream operator overloading makes input and output more convenient.

Q5 – OVERLOADING INCREMENT, COMPARISON AND SUBSCRIPT OPERATORS

5.1 Problem Description

A music player maintains a playlist of tracks and a Time object representing the current playback position. The program must overload prefix and postfix increment operators, comparison operators and the subscript operator.

The playlist must also perform range checking to prevent invalid access.

5.2 Concepts Used

- Prefix increment operator
- Postfix increment operator
- Comparison operators
- Subscript operator
- Const member function
- Function overloading
- Range checking

5.3 Algorithm

1. Start the program.
2. Create a Time class.
3. Store the playback time in seconds.
4. Overload prefix increment to increase the time by one second.
5. Overload postfix increment to return the old value and then increase the time.
6. Overload equality and less-than operators.
7. Create a Playlist class.
8. Store track names in an array.
9. Overload the subscript operator in const and non-const forms.
10. Check whether the requested index is valid.
11. Simulate playback and display the results.
12. Stop the program.

5.4 C++ Program

```
#include <iostream>
#include <string>
#include <stdexcept>
using namespace std;

class Time {
    int seconds;

public:
    Time(int s = 0) : seconds(s) {}

    // Prefix increment
    Time& operator++() {
        ++seconds;
        return *this;
    }

    // Postfix increment
    Time operator++(int) {
        Time old = *this;
        ++seconds;
        return old;
    }
```

```
    bool operator==(const Time& other) const {
        return seconds == other.seconds;
    }

    bool operator<(const Time& other) const {
        return seconds < other.seconds;
    }

    void display() const {
        cout << seconds << " seconds" << endl;
    }
};

class Playlist {
    string tracks[3];

public:
    Playlist() {
        tracks[0] = "Song A";
        tracks[1] = "Song B";
        tracks[2] = "Song C";
    }

    string& operator[](int index) {
        if (index < 0 || index >= 3) {
            throw out_of_range("Invalid playlist index");
        }

        return tracks[index];
    }

    const string& operator[](int index) const {
        if (index < 0 || index >= 3) {
            throw out_of_range("Invalid playlist index");
        }

        return tracks[index];
    }
};

int main() {
    Time t(5);

    Time a = t++;
    Time b = ++t;

    cout << "a = ";
    a.display();

    cout << "b = ";
    b.display();

    cout << "t = ";
    t.display();

    Playlist p;

    cout << p[0] << endl;
    cout << p[1] << endl;

    try {
        cout << p[3] << endl;
    }
    catch (const out_of_range& e) {
        cout << e.what() << endl;
    }
```

```
    return 0;
}
```

5.5 Expected Output

```
a = 5 seconds
b = 7 seconds
t = 7 seconds
Song A
Song B
Invalid playlist index
```

5.7 Predicted Output for the Given Counter Program

Given:

```
Counter c{5};
Counter a = c++;
Counter b = ++c;

cout << a.v << ' ' << b.v << ' ' << c.v << '\n';
```

Step-by-step:

1. Initially, `c.v = 5`.
2. `c++` returns the old value 5 and changes c to 6.
3. `a.v` becomes 5.
4. `++c` changes c from 6 to 7.
5. `b.v` becomes 7.

Expected output:

```
5 7 7
```

5.8 Alternative Concept – at() Function

Instead of using a checked subscript operator, a playlist can provide an `at()` function that throws an exception when an invalid index is used.

An unchecked subscript operator can also be used, but invalid access may cause undefined behaviour.

5.9 Complexity Analysis

Operation	Time Complexity	Space Complexity
Prefix increment	O(1)	O(1)
Postfix increment	O(1)	O(1)
Comparison	O(1)	O(1)
Playlist access	O(1)	O(1)
Range checking	O(1)	O(1)

Both prefix and postfix increment have O(1) time complexity. However, postfix increment creates a temporary copy. Range checking adds a constant-time comparison to playlist access.

5.10 Conclusion

Operator overloading allows time objects and playlists to be accessed naturally. Prefix and postfix operators have different behaviours, and checked subscript access improves program safety by preventing invalid indexing.

Q6 – THE RULES OF OPERATOR OVERLOADING

6.1 Problem Description

An SMS gateway uses a custom Text class to store messages using a dynamically allocated character array. It must support assignment, concatenation, comparison and printing.

The program must correctly manage dynamic memory and prevent self-assignment problems.

6.2 Concepts Used

- Operator overloading
- Copy constructor
- Copy assignment operator
- Destructor
- String concatenation
- Equality and inequality operators
- Stream insertion operator
- Dynamic character array

6.3 Algorithm

1. Start the program.
2. Create a Text class.
3. Declare a character pointer to store the message.
4. Implement a constructor to allocate memory.
5. Implement a copy constructor for deep copying.
6. Implement a copy assignment operator.
7. Check for self-assignment.
8. Overload the addition operator for concatenation.
9. Overload equality and inequality operators.
10. Overload the stream insertion operator.
11. Release memory through the destructor.
12. Test empty messages and long messages.
13. Stop the program.

6.4 C++ Program

```
#include <iostream>
#include <cstring>
using namespace std;

class Text {
    char* data;

public:
    Text(const char* s = "") {
        data = new char[strlen(s) + 1];
        strcpy(data, s);
    }

    // Copy constructor
    Text(const Text& other) {
        data = new char[strlen(other.data) + 1];
        strcpy(data, other.data);
    }

    // Copy assignment
    Text& operator=(const Text& other) {
        if (this != &other) {
            char* temp = new char[strlen(other.data) + 1];
```

```
            strcpy(temp, other.data);

            delete[] data;
            data = temp;
        }

        return *this;
    }

    // Concatenation
    Text operator+(const Text& other) const {
        size_t len = strlen(data) + strlen(other.data) + 1;

        char* temp = new char[len];

        strcpy(temp, data);
        strcat(temp, other.data);

        Text result(temp);

        delete[] temp;

        return result;
    }

    bool operator==(const Text& other) const {
        return strcmp(data, other.data) == 0;
    }

    bool operator!=(const Text& other) const {
        return !(*this == other);
    }

    friend ostream& operator<<(ostream& out,
                               const Text& t) {
        out << t.data;
        return out;
    }

    ~Text() {
        delete[] data;
    }
};

int main() {
    Text a("Hello ");
    Text b("World");

    Text c = a + b;

    cout << "Message: " << c << endl;

    Text d = c;

    cout << "Copied message: " << d << endl;

    cout << boolalpha << (c == d) << endl;

    d = d;

    cout << "After self assignment: " << d << endl;

    return 0;
}
```

6.5 Expected Output

```
Message: Hello World
```

```
Copied message: Hello World
true
After self assignment: Hello World
```

6.6 Explanation

The Text class stores a message using dynamically allocated character memory.

The constructor allocates sufficient memory for the message, including the null terminator. The copy constructor creates a separate character array and copies the message into it.

The copy assignment operator checks for self-assignment before replacing the existing data. It uses a temporary pointer to ensure the old data is not lost if memory allocation fails.

The addition operator combines two messages and returns a new Text object. The equality operator compares two messages, and the stream insertion operator prints the message.

6.7 Operators That Cannot Be Overloaded

The following operators cannot be overloaded:

Operator	Name
`::`	Scope resolution
`.`	Member access
`.*`	Pointer-to-member access
`?:`	Conditional operator
`sizeof`	Size operator

These operators have special language-level meanings and cannot be redefined by user classes.

6.8 Rules of Operator Overloading

1. At least one operand must be a user-defined type.
2. Operator precedence cannot be changed.
3. Operator associativity cannot be changed.
4. The number of operands cannot be changed.
5. New operators cannot be created.
6. Some operators, such as assignment, must be overloaded as member functions.
7. Operator overloading should preserve the expected meaning of the operator.

6.9 Why Must operator= Be a Member Function?

The assignment operator must be a non-static member function because it modifies the left-hand object. The left operand is the object that receives the assignment.

6.10 Alternative Concept – Using std::string

The Text class can be replaced with `std::string`.

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string a = "Hello ";
    string b = "World";

    string c = a + b;
```

```
    cout << c << endl;

    string d = c;

    cout << (c == d) << endl;

    return 0;
}
```

6.11 Complexity Analysis

Let n and m represent the lengths of two messages.

Operation	Time Complexity	Additional Space
Copy constructor	O(n)	O(n)
Copy assignment	O(n)	O(n) temporary
Concatenation	O(n + m)	O(n + m)
Equality comparison	O(min(n,m)) in the worst matching-prefix case	O(1)
Destructor	O(1) to release the allocated buffer	O(1)

For the expression `a + b + c`, the first concatenation copies the characters of a and b. The second concatenation copies the intermediate result and c. Therefore, the total character-copying work is proportional to:

```
(n1 + n2) + (n1 + n2 + n3)
```

Using `std::string` is generally recommended because it manages memory and provides tested string operations.

6.12 Conclusion

Custom operator overloading provides a better understanding of memory management and object behaviour. Deep copying prevents shared-memory errors, while self-assignment checking improves safety. In practical applications, `std::string` is usually preferred because it provides automatic memory management.

Q7 – DESIGN A COMPLEX-NUMBER CLASS FROM FIRST PRINCIPLES

7.1 Problem Description

An electrical engineering laboratory requires a calculator to calculate AC circuit impedance. Electrical impedance is represented using complex numbers.

The calculator must support addition, subtraction, multiplication, division, equality comparison and output formatting. It must also calculate the magnitude of impedance and handle division by zero safely.

7.2 Stage 1 – Problem Analysis

Data Required

1. Real part of impedance.
2. Imaginary part of impedance.
3. Magnitude of impedance.
4. Results of arithmetic operations.
5. Status of division by zero.

Required Operations

- Addition
- Subtraction
- Multiplication
- Division
- Equality comparison
- Magnitude calculation
- Display in rectangular form

Constructors Required

1. Default constructor – initialises both parts to zero.
2. Single-argument constructor – initialises only the real part.
3. Two-argument constructor – initialises real and imaginary parts.

Measurable Requirements

1. The program must create complex numbers using all three constructors.
2. It must perform all required arithmetic operations.
3. It must calculate the magnitude accurately.
4. It must display results in rectangular form.
5. It must prevent division by zero.

7.3 Stage 2 – Class Design

The Complex class contains two private data members:

- `real` – stores the real component.
- `imag` – stores the imaginary component.

The class supports overloaded arithmetic and comparison operators.

Mathematical Formulas

For two complex numbers:

```
Z1 = a + jb

Z2 = c + jd
```

Addition:

```
Z1 + Z2 = (a + c) + j(b + d)
```

Subtraction:

```
Z1 - Z2 = (a - c) + j(b - d)
```

Multiplication:

```
Z1 × Z2 = (ac - bd) + j(ad + bc)
```

Division:

```
Z1 / Z2 = [(ac + bd) / (c² + d²)] + j[(bc - ad) / (c² + d²)]
```

Magnitude:

```
|Z| = sqrt(a² + b²)
```

7.4 Stage 3 – C++ Implementation

```
#include <iostream>
#include <cmath>
#include <stdexcept>
using namespace std;

class Complex {
private:
    double real;
    double imag;

public:
    // Default constructor
    Complex() : real(0), imag(0) {}

    // Constructor with real part
    Complex(double r) : real(r), imag(0) {}

    // Constructor with real and imaginary parts
    Complex(double r, double i)
        : real(r), imag(i) {}

    // Addition
    Complex operator+(const Complex& c) const {
        return Complex(real + c.real,
                       imag + c.imag);
    }

    // Subtraction
    Complex operator-(const Complex& c) const {
        return Complex(real - c.real,
                       imag - c.imag);
    }
```

```
    // Multiplication
    Complex operator*(const Complex& c) const {
        return Complex(
            real * c.real - imag * c.imag,
            real * c.imag + imag * c.real
        );
    }

    // Division
    Complex operator/(const Complex& c) const {
        double denominator =
            c.real * c.real + c.imag * c.imag;

        if (denominator == 0) {
            throw runtime_error("Division by zero impedance");
        }

        return Complex(
            (real * c.real + imag * c.imag) / denominator,
            (imag * c.real - real * c.imag) / denominator
        );
    }

    // Equality
    bool operator==(const Complex& c) const {
        const double eps = 1e-9;

        return fabs(real - c.real) < eps &&
               fabs(imag - c.imag) < eps;
    }

    // Magnitude
    double magnitude() const {
        return sqrt(real * real + imag * imag);
    }

    // Stream output
    friend ostream& operator<<(ostream& out,
                               const Complex& c) {
        out << c.real;

        if (c.imag >= 0) {
            out << " + j" << c.imag;
        } else {
            out << " - j" << -c.imag;
        }

        return out;
    }
};

int main() {
    try {
        Complex z1(3, 4);
        Complex z2(2, 1);

        cout << "Z1 = " << z1 << endl;
        cout << "Z2 = " << z2 << endl;

        cout << "Addition = " << z1 + z2 << endl;
        cout << "Subtraction = " << z1 - z2 << endl;
        cout << "Multiplication = " << z1 * z2 << endl;
        cout << "Division = " << z1 / z2 << endl;

        cout << "Magnitude of Z1 = "
             << z1.magnitude() << endl;
```

```
        Complex zero(0, 0);

        cout << "Division by zero test:" << endl;
        cout << z1 / zero << endl;
    }
    catch (const exception& e) {
        cout << "Error: " << e.what() << endl;
    }

    return 0;
}
```

7.5 Expected Output

```
Z1 = 3 + j4
Z2 = 2 + j1
Addition = 5 + j5
Subtraction = 1 + j3
Multiplication = 2 + j11
Division = 2 + j1
Magnitude of Z1 = 5
Division by zero test:
Error: Division by zero impedance
```

7.6 Hand Calculation and Verification

Given:

$Z1 = 3 + j4$

$Z2 = 2 + j1$

Addition:

$Z1 + Z2 = (3 + 2) + j(4 + 1)$

$= 5 + j5$

Subtraction:

$Z1 - Z2 = (3 - 2) + j(4 - 1)$

$= 1 + j3$

Multiplication:

$Z1 \times Z2 = (3 \times 2 - 4 \times 1) + j(3 \times 1 + 4 \times 2)$

$= 2 + j11$

Division:

Denominator:

$2^2 + 1^2 = 5$

Real part:

$(3 \times 2 + 4 \times 1) / 5 = 10 / 5 = 2$

Imaginary part:

$(4 \times 2 - 3 \times 1) / 5 = 5 / 5 = 1$

Therefore:

```
Z1 / Z2 = 2 + j1
```

Magnitude:

```
|Z1| = sqrt(3² + 4²)

= sqrt(25) = 5
```

The program output matches the hand calculations.

7.7 Series and Parallel Impedance

For series-connected impedances:

```
Zseries = Z1 + Z2
```

For parallel-connected impedances:

```
Zparallel = (Z1 × Z2) / (Z1 + Z2)
```

The overloaded arithmetic operators allow these formulas to be implemented directly.

Example:

```
Complex series = z1 + z2;
Complex parallel = (z1 * z2) / (z1 + z2);

cout << "Series impedance: " << series << endl;
cout << "Parallel impedance: " << parallel << endl;
```

7.8 Division by Zero Handling

Division by zero occurs when the denominator impedance is zero.

The program checks whether:

```
c.real * c.real + c.imag * c.imag == 0
```

If the denominator is zero, a runtime exception is thrown. The catch block displays an error message instead of allowing an invalid calculation.

For practical numerical applications, a suitable tolerance may be used to detect values very close to zero.

7.9 Stage 4 – Design Evaluation

Comparison with std::complex<double>

Custom Complex Class	std::complex<double>
Designed by the programmer	Provided by the C++ standard library
Requires manual operator implementation	Operators are already implemented
Useful for understanding operator overloading	Suitable for practical applications
Requires manual division-by-zero policy	Provides standard complex arithmetic behaviour
Requires testing and maintenance	Well-established library implementation

Why Each Constructor Is Needed

- Default constructor creates a complex number equal to zero.
- Single-argument constructor creates a real number with zero imaginary part.
- Two-argument constructor creates a complete complex number.

Remaining Limitation

The implementation uses floating-point values, so very large, very small or nearly equal values may introduce numerical precision errors. The equality operator uses a tolerance to reduce problems caused by floating-point representation.

7.10 Alternative Concept – Using std::complex

```
#include <iostream>
#include <complex>
using namespace std;

int main() {
    complex<double> z1(3, 4);
    complex<double> z2(2, 1);

    cout << "Addition: " << z1 + z2 << endl;
    cout << "Subtraction: " << z1 - z2 << endl;
    cout << "Multiplication: " << z1 * z2 << endl;
    cout << "Division: " << z1 / z2 << endl;

    cout << "Magnitude: " << abs(z1) << endl;

    return 0;
}
```

7.11 Complexity Analysis

Operation	Time Complexity	Space Complexity
Addition	O(1)	O(1)
Subtraction	O(1)	O(1)
Multiplication	O(1)	O(1)
Division	O(1)	O(1)
Equality	O(1)	O(1)
Magnitude	O(1)	O(1)

Both the custom class and `std::complex<double>` use constant time and constant space for individual arithmetic operations. The standard library implementation is generally preferable in real applications because it is already tested and maintained.

7.12 Conclusion

The Complex class demonstrates how constructors and operator overloading can represent mathematical concepts in C++. It performs arithmetic operations, calculates magnitude and safely handles division by zero. Comparing the custom class with `std::complex<double>` shows the importance of understanding the concepts while also using standard library components in practical development.

OVERALL COMPARISON OF CO3 CONCEPTS

Question	Main Concept	Application
Q1	Constructor overloading	Library book records
Q2	Deep copy and Rule of Three	Hospital prescriptions
Q3	Dynamic constructor and destructor	Auditorium seat booking
Q4	Arithmetic and stream operators	Money billing system

Q5	Increment, comparison and subscript operators	Music playlist
Q6	Operator overloading rules	SMS text management
Q7	Complex class design	AC circuit impedance

OVERALL CONCLUSION

This assignment demonstrates the use of constructors, destructors and operator overloading in C++. Different constructors allow objects to be created with different initial values. Copy constructors and copy assignment operators help create independent objects when dynamic memory is used. Destructors release memory automatically when objects are destroyed.

Operator overloading makes user-defined classes easier to work with by supporting familiar operations such as addition, subtraction, comparison, increment and stream input/output. The assignment also demonstrates the importance of deep copying, self-assignment checks, range validation and safe memory management.

The alternative implementations show that standard library components such as `std::vector`, `std::string` and `std::complex` can reduce manual memory-management work. Overall, these concepts help develop reusable, efficient and reliable C++ applications.

REFERENCES

1. Bjarne Stroustrup, *The C++ Programming Language*, 4th Edition, Addison-Wesley.
2. Stanley B. Lippman, Josée Lajoie and Barbara E. Moo, *C++ Primer*, 5th Edition, Addison-Wesley.
3. Scott Meyers, *Effective C++*, 3rd Edition, Addison-Wesley.
4. cppreference.com, “Constructors and Destructors in C++.”
5. cppreference.com, “Copy Constructors and Copy Assignment Operators.”
6. cppreference.com, “Operator Overloading.”
7. ISO C++ Documentation, “C++ Language Reference.”
8. GeeksforGeeks, “Constructors in C++.”
9. GeeksforGeeks, “Shallow Copy and Deep Copy in C++.”
10. LearnCpp.com, “Object-Oriented Programming and Operator Overloading.”